UD2 · Sintaxis de JavaScript

Módulo 0612 · Desarrollo Web en Entorno Cliente (dew) CFGS Desarrollo de Aplicaciones Web · 2.º curso · Curso 2026/27

Resultado de aprendizaje	RA2. Escribe sentencias simples, aplicando la sintaxis del lenguaje y verificando su ejecución sobre navegadores web.
Duración	15 horas (semanas 2 a 4)
Hito de la unidad	Escribes programas pequeños pero completos en JavaScript, los pruebas en el navegador, los depuras con las DevTools y los documentas con JSDoc. Tu proyecto ya calcula cosas sobre los datos de tu tema.

Qué vas a aprender

Al terminar esta unidad serás capaz de:

- Ejecutar código JavaScript en la consola del navegador, en un fichero enlazado a una página y en Node.js, y saber cuándo conviene cada opción.
- Declarar variables y constantes con `let` y `const`, elegir nombres claros y entender por qué `var` ha quedado en desuso.
- Explicar dónde «vive» una variable (ámbito global, de función o de bloque) y qué es el *hoisting*.
- Distinguir los tipos de datos del lenguaje y predecir qué ocurre cuando JavaScript convierte un tipo en otro.
- Usar los operadores aritméticos, de comparación, lógicos, de asignación y los más recientes (`??`, `?.`).
- Tomar decisiones con `if`, `else if` y `switch`, y repetir tareas con `while`, `do…while`, `for` y `for…of`.
- Comentar y documentar el código con JSDoc, y mantener un estilo uniforme con Prettier y ESLint.
- Localizar y corregir errores con la consola y el depurador de las DevTools, y probar tu código de forma sistemática.

Criterios de evaluación (RA2)

CE	Se evaluará si…
a	Has seleccionado un lenguaje de programación de clientes web en función de sus posibilidades.
b	Has utilizado los distintos tipos de variables y operadores disponibles en el lenguaje.
c	Has identificado los ámbitos de utilización de las variables.
d	Has reconocido y comprobado las peculiaridades del lenguaje respecto a las conversiones entre distintos tipos de datos.
e	Has utilizado mecanismos de decisión en la creación de bloques de sentencias.
f	Has utilizado bucles y has verificado su funcionamiento.
g	Has añadido comentarios al código.
h	Has utilizado herramientas y entornos para facilitar la programación, prueba y documentación del código.

1. Dónde y cómo ejecutamos JavaScript

1.1. El lenguaje que hemos elegido y por qué

En la UT1 viste que el navegador solo ejecuta **JavaScript** (y lo que se traduce a él, como TypeScript). No hay alternativa real en el cliente: es el lenguaje de la web y el que usaremos durante todo el curso. Lo que sí podemos elegir es **qué versión** del lenguaje usamos. En este módulo trabajaremos con **JavaScript moderno** (ES2015 en adelante): `let` y `const`, plantillas literales, funciones flecha, módulos… Es lo que entienden todos los navegadores actuales y lo que usa Angular. Cuando veas en internet código con `var` por todas partes, sospecha: probablemente sea anterior a 2015.

1.2. Tres formas de ejecutar código

Entorno	Cómo	Para qué sirve	Limitaciones
Consola de las DevTools	F12 → pestaña **Console**	Probar una expresión, inspeccionar un valor, experimentar	El código se pierde al recargar; no se guarda en el repositorio

Fichero `.js` enlazado a una página	`<script src="js/app.js" defer>` y **Live Server**	Programas completos; es como funciona la web de verdad	Hay que recargar para ver los cambios (Live Server lo hace solo)
Node.js	`node fichero.js` en la terminal	Ejecutar scripts sin navegador; herramientas de desarrollo	No hay `document`, `window`, `alert` ni `prompt`

En esta unidad usaremos las tres: la **consola** para experimentar, un **fichero enlazado** para los programas que entregas y **Node.js** en algún momento para ver que el lenguaje es el mismo dentro y fuera del navegador.

> ***Consejo.*** *En la consola puedes escribir varias líneas pulsando* ***Mayús + Intro****. Con la flecha arriba recuperas lo que ya escribiste. Y si quieres guardar un experimento, en el panel* ***Sources*** *→* ***Snippets*** *puedes crear fragmentos de código con nombre y ejecutarlos con* ***Ctrl + Intro****.*

1.3. La consola como herramienta de salida

Durante esta unidad, la **consola** será la «pantalla» de tus programas. `console` es un objeto que el navegador (y Node.js) pone a tu disposición con varios métodos:

```
console.log('Mensaje normal');           // lo más habitual
console.log('Varios', 'valores', 42);    // los separa con espacios
console.info('Información');
console.warn('Aviso: esto no es un error, pero fíjate');
console.error('Error: algo ha ido mal');
console.table([{ nombre: 'Ada', edad: 36 }, { nombre: 'Linus', edad: 56 }]);
console.group('Detalles');               // agrupa los mensajes siguientes
console.log('dentro del grupo');
console.groupEnd();
console.clear();                          // limpia la consola
```

Fíjate en que `console.log` con varios argumentos muestra cada valor tal cual es: un número se ve como número y un texto como texto. Más adelante verás que es útil para saber de qué tipo es cada cosa.

1.4. Entrada de datos: `prompt` (de momento)

Todavía no sabemos leer formularios (eso llega en la UT7). Para que nuestros programas reciban datos usaremos `prompt`, que abre un cuadro de diálogo del navegador y **devuelve siempre un texto** (o `null` si se pulsa Cancelar):

```
const nombre = prompt('¿Cómo te llamas?');
const edadTexto = prompt('¿Cuántos años tienes?'); // '17', un texto
const edad = Number(edadTexto);                    // 17, un número
console.log(`Hola, ${nombre}. El año que viene tendrás ${edad + 1}.`);
```

`prompt` no existe en Node.js y en una aplicación real no se usa: es tosco y bloquea la página. Lo utilizamos aquí solo como herramienta de aprendizaje. El apartado 5 explica por qué es imprescindible convertir lo que devuelve.

1.5. Modo estricto

JavaScript arrastra decisiones de 1995 que hoy se consideran errores de diseño. El **modo estricto** (`'use strict'`) desactiva las más peligrosas: por ejemplo, prohíbe crear variables sin declararlas. Se activa escribiendo esa cadena en la primera línea de un fichero o de una función:

```
'use strict';

total = 10; // ReferenceError: total is not defined
```

Sin modo estricto, esa línea crearía en silencio una variable global llamada `total`, y una errata en un nombre (`totla = 10`) pasaría desapercibida. Los **módulos** (`<script type="module">`) y las **clases** funcionan siempre en modo estricto, así que en Angular no tendrás que escribirlo. En esta unidad lo pondremos al principio de cada fichero para acostumbrarnos a un JavaScript sin sorpresas.

1.6. Sentencias, bloques y punto y coma

Un programa es una secuencia de **sentencias**, que normalmente terminan en **punto y coma**. JavaScript los añade automáticamente si faltan (*Automatic Semicolon Insertion*), pero esa regla tiene excepciones que producen errores muy difíciles de ver. La convención de este módulo es **escribir siempre el punto y coma**; Prettier lo hará por ti.

Un **bloque** es un conjunto de sentencias entre llaves `{ }`. Los usan las decisiones, los bucles y las funciones, y, como verás en el apartado 3, definen un ámbito propio para `let` y `const`.

```
{
  const mensaje = 'Estoy dentro de un bloque';
  console.log(mensaje);
}
```

1.7. Estructura de trabajo de la unidad

Todos los ejercicios de esta unidad van en tu repositorio `dew-ejercicios`, en la carpeta `ut02`. Cada actividad es una carpeta con su `index.html` mínimo y su fichero `.js`:

```
dew-ejercicios/
└── ut02/
    ├── a1-consola/
    │   ├── index.html
    │   └── app.js
    ├── a2-variables/
    └── ...
```

Y este es el `index.html` mínimo, que solo sirve para cargar el script (el HTML no cambia en toda la unidad):

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>UT2 · A1</title>
  <script src="app.js" defer></script>
</head>
<body>
  <h1>UT2 · A1 · Consola</h1>
  <p>Abre la consola (F12) para ver el resultado.</p>
</body>
</html>
```

2. Variables y constantes

2.1. Declarar con `let` y `const`

Una **variable** es un nombre que se asocia a un valor. En JavaScript moderno se declara con `let` (si el valor va a cambiar) o `const` (si no va a cambiar):

```javascript
let contador = 0;           // se podrá modificar
contador = contador + 1;    // 1
contador += 1;              // 2

const IGIC = 0.07;          // no se podrá reasignar
IGIC = 0.09;                // TypeError: Assignment to constant variable.

let sinValor;               // declarada pero sin valor: undefined
console.log(sinValor);      // undefined
```

`const` obliga a dar un valor en el momento de declarar, y ese nombre no podrá apuntar a otro valor después. Cuidado con una confusión frecuente: `const` **no hace inmutable el valor**, solo impide reasignar el nombre. Un objeto o un array declarados con `const` pueden modificarse por dentro (lo verás en la UT4).

Regla del módulo: usa `const` por defecto y cambia a `let` solo cuando necesites reasignar. Así, quien lea tu código sabe de un vistazo qué valores cambian y cuáles no.

2.2. ¿Y `var`?

Hasta 2015 la única forma de declarar variables era `var`. Sigue funcionando, pero tiene dos comportamientos que provocan errores: **no respeta los bloques** (una `var` declarada dentro de un `if` o de un `for` existe en toda la función) y **se puede redeclarar** sin que nadie avise. Lo verás con detalle en el apartado 3. La conclusión es simple: **en código nuevo no se usa `var`**. Tienes que reconocerlo para leer código antiguo, nada más.

	`var`	`let`	`const`
Ámbito	Función	Bloque	Bloque
¿Se puede reasignar?	Sí	Sí	No
¿Se puede redeclarar en el mismo ámbito?	Sí	No	No
¿Se puede usar antes de la declaración?	Sí (vale `undefined`)	No (error)	No (error)
Uso hoy	Código antiguo	Valores que cambian	Todo lo demás

2.3. Nombres de variables

Las reglas del lenguaje son pocas: un nombre puede contener letras, dígitos, `_` y `$`; no puede empezar por dígito; distingue mayúsculas de minúsculas (`precio` y `Precio` son variables distintas) y no puede ser una **palabra reservada** (`let`, `for`, `class`, `new`…). Las **convenciones** que seguiremos son las habituales en JavaScript:

Qué	**Convención**	**Ejemplos**
Variables y funciones	`camelCase`, empezando en minúscula	`precioTotal`, `numeroDeIntentos`, `calcularMedia`
Constantes «de verdad» (valores fijos conocidos de antemano)	`MAYUSCULAS_CON_GUIONES`	`IGIC`, `MAX_INTENTOS`, `DIAS_SEMANA`
Clases (UT4)	`PascalCase`	`Receta`, `RutaSenderismo`
Booleanos	Empiezan por `es`, `tiene`, `hay`, `puede`	`esValido`, `tieneDescuento`, `hayErrores`

Y sobre todo: **nombres significativos**, o sea, que **expliquen qué contienen**. `precioConIgic` es mejor que `p2`, y `numeroDeAlumnos` mejor que `n`. Evita tildes y eñes en los nombres: el lenguaje las admite, pero dan problemas con algunas herramientas y no es la costumbre en la profesión.

2.4. *Hoisting*: las declaraciones «suben»

Antes de ejecutar un fichero o una función, el motor de JavaScript lo recorre y **registra todas las declaraciones**. A este comportamiento se le llama *hoisting* («elevación»). Sus consecuencias dependen de cómo se declaró la variable:

```
console.log(a); // undefined (¡no da error!)
var a = 1;

console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 2;
```

Con `var`, la declaración sube al principio con valor `undefined`, así que el código anterior es válido aunque no tenga sentido. Con `let` y `const`, la variable también se registra, pero **no se puede tocar hasta llegar a su línea**: ese tramo se conoce como **zona muerta temporal** (*temporal dead zone*, TDZ). Es el comportamiento razonable: usar una variable antes de declararla es casi siempre un error, y `let` y `const` lo convierten en un error visible.

3. Ámbitos de las variables

El **ámbito** (*scope*) de una variable es la región del programa en la que existe y puede usarse. En JavaScript hay tres:

3.1. Ámbito global

Lo que declaras fuera de cualquier función o bloque es **global**: se puede usar desde cualquier parte del fichero. Con `<script>` clásicos, además, las variables globales de un fichero son visibles desde todos los demás scripts de la página, y las declaradas con `var` se convierten en propiedades del objeto `window`. Esto provoca choques de nombres cuando la aplicación crece, y por eso los módulos (UT4) tienen cada uno su propio ámbito.

```
const app = 'Recetario';   // global
var version = 2;            // global y, además, window.version

function mostrar() {
  console.log(app, version); // puede usarlas
}
```

3.2. Ámbito de función

Lo que declaras dentro de una función solo existe dentro de ella. Cada llamada crea sus variables de nuevo.

```
function calcularTotal() {
  const subtotal = 100;          // solo existe aquí dentro
  return subtotal * (1 + IGIC);
}
console.log(subtotal); // ReferenceError: subtotal is not defined
```

3.3. Ámbito de bloque

Desde ES2015, `let` y `const` crean variables que solo existen dentro del **bloque** `{ }` en el que se declaran: el cuerpo de un `if`, de un `for`, de un `while`… `var` **no** respeta este ámbito.

```
if (true) {
  let dentro = 'con let';
  var fuera = 'con var';
}
console.log(fuera);  // 'con var'  → var se ha escapado del bloque
console.log(dentro); // ReferenceError → let se ha quedado dentro
```

El caso más famoso es el contador de un bucle:

```
for (var i = 0; i < 3; i++) {}
console.log(i); // 3  → i sigue existiendo después del bucle

for (let j = 0; j < 3; j++) {}
console.log(j); // ReferenceError → j solo existía dentro del for
```

3.4. Ámbitos anidados y sombreado

Los ámbitos se anidan: desde un bloque interior se ven las variables de los bloques exteriores, pero no al revés. Si en un ámbito interior declaras una variable **con el mismo nombre** que otra exterior, la interior la «tapa» mientras dura el bloque. Se llama **sombreado** (*shadowing*) y, aunque es legal, confunde: evítalo eligiendo nombres distintos.

```
const nivel = 'exterior';
{
  const nivel = 'interior'; // sombrea a la exterior
  console.log(nivel);       // 'interior'
}
console.log(nivel);         // 'exterior'
```

> ***Importante.*** *Tres normas que evitan la mayoría de los problemas de ámbito: declara siempre las variables (el modo estricto te obliga), declara cada variable* ***lo más cerca posible de donde se usa*** *y* ***no reutilices nombres*** *en ámbitos anidados. En el depurador de las DevTools (apartado 11), el panel* ***Scope*** *te muestra en cada momento qué variables existen en cada ámbito.*

4. Tipos de datos

4.1. Tipado dinámico

En Java, cada variable tiene un tipo fijo: `int edad`. En JavaScript, **las variables no tienen tipo; los valores sí**. Una misma variable puede guardar primero un número y luego un texto:

```
let dato = 42;        // ahora contiene un number
dato = 'cuarenta y dos'; // ahora contiene un string (legal, pero mala idea)
```

El operador `typeof` devuelve el tipo del valor actual como un texto:

```
typeof 42;          // 'number'
typeof 'hola';      // 'string'
typeof true;        // 'boolean'
typeof undefined;   // 'undefined'
typeof null;        // 'object'  ← error histórico del lenguaje, nunca corregido
typeof 10n;         // 'bigint'
typeof {};          // 'object'
typeof [];          // 'object'  ← los arrays son objetos
typeof console.log; // 'function'
```

4.2. Tipos primitivos y tipos de referencia

JavaScript tiene **siete tipos primitivos** y un tipo de **referencia** (los objetos, que incluyen arrays y funciones). La diferencia práctica: un primitivo se copia por valor; un objeto se comparte por referencia (dos variables pueden apuntar al mismo objeto). Los objetos se estudian en la UT4; en esta unidad trabajamos sobre todo con primitivos.

Tipo	Qué guarda	Ejemplos
`number`	Números enteros y decimales	`42`, `-3.5`, `1e6`, `NaN`, `Infinity`
`bigint`	Enteros de tamaño arbitrario	`9007199254740993n`
`string`	Texto	`'hola'`, `"hola"`, `` `hola` ``
`boolean`	Verdadero o falso	`true`, `false`
`undefined`	«Sin valor todavía»	Una variable declarada sin asignar
`null`	«Sin valor, a propósito»	`let seleccionado = null;`
`symbol`	Identificadores únicos (uso avanzado)	`Symbol('id')`
`object`	Colecciones de datos y funciones	`{ nombre: 'Ada' }`, `[1, 2, 3]`, `function () {}`

4.3. Números

Hay **un solo tipo numérico**, `number`, para enteros y decimales, basado en el estándar IEEE 754 de 64 bits (como el `double` de Java). Eso tiene consecuencias que conviene conocer:

```
0.1 + 0.2;                  // 0.30000000000000004  → los decimales no son exactos
0.1 + 0.2 === 0.3;          // false
(0.1 + 0.2).toFixed(2);     // '0.30'  → redondea a texto con 2 decimales
Number.MAX_SAFE_INTEGER;    // 9007199254740991 → por encima, usa bigint

10 / 0;                     // Infinity (no da error)
-10 / 0;                    // -Infinity
'hola' * 2;                 // NaN ("Not a Number"): el resultado de una operación
imposible
NaN === NaN;                // false (!)  → NaN no es igual ni a sí mismo
Number.isNaN('hola' * 2);   // true  → así se comprueba
```

Para dinero, la práctica habitual es trabajar en **céntimos con enteros** o redondear solo al mostrar. Otros valores y funciones útiles: `Math.round`, `Math.floor`, `Math.ceil`, `Math.trunc`, `Math.random()` (decimal entre 0 y 1), `Math.max`, `Math.min` y `Math.pow` (o el operador `**`). El objeto `Math` se estudia en la UT3; aquí usaremos solo estas.

Se pueden escribir números con separadores para leerlos mejor: `1_000_000`. Y con notación científica: `1e6` es un millón.

4.4. Cadenas de texto

Un `string` es una secuencia de caracteres Unicode. Se escribe entre comillas simples, dobles o acentos graves (plantillas literales). La convención del módulo es **comillas simples** para cadenas normales y **plantillas** cuando hay que insertar valores.

```
const nombre = 'Ada';
const saludo = "Hola, " + nombre + "."; // concatenación con +
const saludo2 = `Hola, ${nombre}.`;      // plantilla literal: interpolación con
${ }

const multilinea = `Primera línea
segunda línea`;                          // las plantillas admiten saltos de
línea

'Con \'comilla\' escapada';              // \ escapa el carácter siguiente
'Salto\nde línea y tab\tulador';
```

Dentro de `${ }` puede ir **cualquier expresión**: `${precio * 1.07}`, `${edad >= 18 ? 'mayor' : 'menor'}`.

Algunas operaciones básicas que necesitarás ya (el objeto `String` completo se ve en la UT3):

```
const texto = 'Desarrollo Web';
texto.length;              // 15  → número de caracteres
texto[0];                  // 'D' → acceso por posición (empieza en 0)
texto.at(-1);              // 'b' → último carácter
texto.toUpperCase();       // 'DESARROLLO WEB'
texto.toLowerCase();       // 'desarrollo web'
texto.includes('Web');     // true
texto.startsWith('Des');   // true
texto.indexOf('Web');      // 11
texto.slice(0, 10);        // 'Desarrollo'
'  espacios  '.trim();     // 'espacios'
'a-b-c'.split('-');        // ['a', 'b', 'c']  (un array, UT4)
'ab'.repeat(3);            // 'ababab'
```

Las cadenas son **inmutables**: ningún método las cambia; todos devuelven una cadena nueva. `texto.toUpperCase()` no modifica `texto`; si quieres conservar el resultado, guárdalo: `const mayusculas = texto.toUpperCase();`.

4.5. Booleanos

Solo dos valores, `true` y `false`. Son el resultado de las comparaciones y lo que evalúan las decisiones y los bucles. No los compares con `true`: `if (esValido === true)` se escribe `if (esValido)`.

4.6. `undefined` y `null`

Ambos representan «ausencia de valor», pero con matices:

- `undefined` lo pone el lenguaje: una variable declarada sin valor, un parámetro que no se pasó, una propiedad que no existe, una función que no devuelve nada.
- `null` lo pones tú, para decir explícitamente «aquí no hay nada» (por ejemplo, «todavía no se ha seleccionado ningún elemento»). `prompt` devuelve `null` si el usuario cancela.

```
let seleccion;             // undefined: aún no se ha asignado
seleccion = null;          // null: se ha decidido que no hay selección
seleccion === undefined;   // false
seleccion == null;         // true  (== considera iguales null y undefined)
```

5. Conversiones entre tipos

Aquí está la principal «peculiaridad» de JavaScript respecto a Java: como es un lenguaje de **tipado débil**, cuando una operación recibe un tipo que no espera, **convierte el valor en silencio** en lugar de dar error. A esa conversión automática se la llama **coerción**. Entenderla es la diferencia entre un programa que funciona y uno que muestra «NaN €» al usuario.

5.1. Conversión explícita: la que decides tú

Las funciones `Number()`, `String()` y `Boolean()` convierten cualquier valor al tipo correspondiente. Es la forma recomendada de convertir: se ve en el código lo que ocurre.

A número

```
Number('42');       // 42
Number('3.14');     // 3.14
Number('  7  ');    // 7  → ignora espacios
Number('');         // 0  (!)
Number('42px');     // NaN → no admite texto sobrante
Number('hola');     // NaN
Number(true);       // 1
Number(false);      // 0
Number(null);       // 0
Number(undefined);  // NaN

parseInt('42px');   // 42  → lee dígitos hasta el primer carácter no válido
parseInt('3.99');   // 3   → entero: trunca
parseFloat('3.99'); // 3.99
parseInt('hola');   // NaN
parseInt('101', 2); // 5   → segundo argumento: base numérica
```

Regla práctica: usa `Number()` para convertir lo que escribe el usuario, y `parseInt`/`parseFloat` solo cuando **quieras** ignorar el texto sobrante (`'12px'`). El operador unario `+` (`+'42'`) equivale a `Number('42')`; funciona, pero es menos legible.

A texto

```
String(42);          // '42'
String(true);        // 'true'
String(null);        // 'null'
(3.14159).toFixed(2); // '3.14' → redondea a n decimales y devuelve texto
(255).toString(16);   // 'ff'   → en otra base
```

A booleano

```
Boolean(0);          // false
Boolean('');         // false
Boolean('0');        // true  (!) → una cadena no vacía siempre es true
Boolean(' ');        // true
Boolean(null);       // false
Boolean(undefined);  // false
Boolean(NaN);        // false
Boolean(42);         // true
Boolean('false');    // true  (!)
Boolean([]);         // true  → cualquier objeto es true, aunque esté vacío
```

5.2. Valores *falsy* y *truthy*

Solo **siete valores** se convierten en `false`: `false`, `0`, `-0`, `0n`, `''` (cadena vacía), `null`, `undefined` y `NaN`. Se los llama ***falsy***. **Todo lo demás** es ***truthy***, incluidos `'0'`, `'false'`, `[]` y `{}`. Esta lista es la que aplica `if` cuando la condición no es un booleano:

```
const nombre = prompt('Nombre:');
if (nombre) {                // true si no es '' ni null
  console.log(`Hola, ${nombre}`);
} else {
  console.log('No has escrito nada');
}
```

5.3. Conversión implícita: la que hace el lenguaje

El operador `+` es el más traicionero: si **alguno** de los dos operandos es una cadena, concatena; si no, suma.

```
'5' + 2;      // '52'  → concatena
5 + 2;        // 7
'5' + '2';    // '52'
1 + 2 + '3';  // '33'  → (1 + 2) = 3, luego 3 + '3' = '33'
'1' + 2 + 3;  // '123' → de izquierda a derecha, todo concatena
```

El mayor riesgo viene cuando este valor es extraído de un formulario o un parámetro obtenido desde la web. Estos valores son típicamente cadenas de caracteres, y, si no lo verificamos antes, el resultado del operador `+` acabará siendo siempre una concatenación.

Los demás operadores aritméticos (`-`, `*`, `/`, `%`, `**`) solo saben trabajar con números, así que convierten las cadenas a número:

```
'5' - 2;      // 3
'5' * '2';    // 10
'10' / '4';   // 2.5
'5' - 'a';    // NaN
true + 1;     // 2   → true se convierte en 1
null + 1;     // 1   → null se convierte en 0
undefined + 1; // NaN
```

El error clásico con `prompt`:

```
const edad = prompt('Edad:');   // '17' (texto)
console.log(edad + 1);          // '171'  ← concatenación
console.log(Number(edad) + 1);  // 18     ← lo que queríamos
```

> ***Importante.*** *Todo lo que llega del usuario (`prompt`, y en el futuro los formularios y la URL)* ***es texto****. Conviértelo con `Number()` antes de operar, y comprueba con `Number.isNaN()` que la conversión ha funcionado.*

5.4. Comparar: `==` frente a `===`

JavaScript tiene dos operadores de igualdad:

- `==` (**igualdad débil**): antes de comparar, **convierte** los operandos a un tipo común. Sus reglas son complicadas y producen resultados sorprendentes.
- `===` (**igualdad estricta**): compara **sin convertir**. Si los **tipos** son distintos, el resultado es `false`.

```
5 == '5';          // true   → convierte '5' a 5
5 === '5';         // false  → tipos distintos
0 == '';           // true   (!)
0 == '0';          // true   (!)
'' == '0';         // false  (!!) → == no es ni siquiera transitivo
null == undefined; // true
null === undefined; // false
false == '0';      // true   (!)
NaN == NaN;        // false
```

Regla del módulo: usa siempre `===` y `!==`. ESLint te avisará si escribes `==`. La única excepción tolerada es `valor == null`, que comprueba a la vez `null` y `undefined`; aun así, es más claro escribir las dos comparaciones.

5.5. Tabla resumen de conversiones

Valor	Number()	String()	Boolean()
''	0	''	false
'42'	42	'42'	true
'hola'	NaN	'hola'	true
'0'	0	'0'	true
0	0	'0'	false
42	42	'42'	true
NaN	NaN	'NaN'	false
true	1	'true'	true
false	0	'false'	false
null	0	'null'	false
undefined	NaN	'undefined'	false

6. Operadores

Un **operador** combina uno o más valores (**operandos**) y produce un resultado. Una combinación de valores, variables y operadores que produce un valor es una **expresión**.

6.1. Aritméticos

Operador	Nombre	Ejemplo	Resultado
`+`	Suma (o concatenación)	`7 + 2`	`9`
`-`	Resta	`7 - 2`	`5`
`*`	Multiplicación	`7 * 2`	`14`
`/`	División (siempre decimal)	`7 / 2`	`3.5`
`%`	Resto de la división entera	`7 % 2`	`1`
`**`	Potencia	`7 ** 2`	`49`
`++`	Incremento en 1	`i++`	`i` + 1
`--`	Decremento en 1	`i--`	`i` - 1
`-` (unario)	Cambio de signo	`-x`	

El **resto** (`%`) es más útil de lo que parece: `n % 2 === 0` dice si `n` es par; `n % 10` da la última cifra; `minutos % 60` convierte minutos en «minutos que sobran tras las horas». Con `Math.trunc(7 / 2)` obtienes la **división entera** (`3`).

`i++` y `++i` incrementan `i`, pero como expresión valen distinto: `i++` devuelve el valor antiguo, `++i` el nuevo.

```
let a = 3;
let b = a++;
let c = ++a;

console.log(b);          // b devuelve 3 (el valor de a antes del ++)
console.log(c);          // c devuelve 4 (el valor de a después del ++
```

6.2. Asignación y asignación compuesta

El operador `=` asigna. Los operadores compuestos abrevian «operar y guardar en la misma variable»:

```
let total = 10;
total += 5;   // total = total + 5  → 15
total -= 3;   // 12
total *= 2;   // 24
total /= 4;   // 6
total %= 4;   // 2
total **= 3;  // 8

let texto = 'Hola';
texto += ', mundo'; // 'Hola, mundo' → también concatena
```

6.3. Comparación

Devuelven siempre un booleano.

Operador `===`	Significado Igual (estricto)	Ejemplo `5 === 5`	Resultado `true`
`!==`	Distinto (estricto)	`5 !== '5'`	`true`
`==`, `!=`	Igual y distinto con conversión	`5 == '5'`	`true` (evitar)
`>`, `<`	Mayor, menor	`7 > 2`	`true`
`>=`, `<=`	Mayor o igual, menor o igual	`7 <= 7`	`true`

Las cadenas se comparan **carácter a carácter según su código Unicode**, no alfabéticamente como en un diccionario: `'Zapato' < 'avión'` es `true` porque las mayúsculas van antes que las minúsculas, y `'10' < '9'` es `true` porque `'1'` va antes que `'9'`. Compara números como números (convierte primero) y, para ordenar textos con criterios de idioma, usa `localeCompare` (UT3).

6.4. Lógicos

Operador `&&`	Nombre Y (AND)	`true` cuando… Los dos operandos son *truthy*
`||`	O (OR)	Al menos uno es *truthy*
`!`	NO (NOT)	El operando es *falsy*

```
const edad = 20;
const tieneCarnet = true;
edad >= 18 && tieneCarnet;        // true
edad < 18 || !tieneCarnet;        // false
!(edad >= 18);                    // false
```

Dos detalles importantes:

Evaluación de ciclo corto. En JavaScript, `&&` deja de evaluar en cuanto encuentra un operando *falsy*, y `||` en cuanto encuentra uno *truthy*. El segundo operando **no llega a ejecutarse** si no hace falta. Se aprovecha para escribir condiciones seguras: `n !== 0 && total / n > 2` nunca llega a la operación `total / n` porque `n !== 0` ya resuelve la condición, por lo que nunca se produce la división entre cero y, por tanto, no se genera un error.

Devuelven un operando, no un booleano. `&&` y `||` devuelven el **valor** del operando que decidió el resultado:

```
'hola' || 'por defecto';   // 'hola'
'' || 'por defecto';       // 'por defecto'  → clásico para valores por defecto
0 || 10;                   // 10  (!) → pero 0 podía ser un valor válido...
'hola' && 'otro';          // 'otro'
```

Ese «`0 || 10` devuelve 10» es un problema cuando `0` o `''` son valores legítimos. Para eso existe el operador siguiente.

6.5. Coalescencia nula `??` y encadenamiento opcional `?.`

`??` devuelve el operando derecho **solo si** el izquierdo es `null` o `undefined`. A diferencia de `||`, respeta `0`, `''` y `false`:

```
const unidades = 0;
unidades || 1;    // 1  → || trata el 0 como «nada»
unidades ?? 1;    // 0  → ?? solo sustituye null y undefined

const nombre = prompt('Nombre:') ?? 'anónimo'; // si cancela (null), 'anónimo'
```

Existe también la asignación `??=`: `config.idioma ??= 'es'` asigna solo si estaba en `null` o `undefined`.

`?.` accede a una propiedad **solo si** el valor de la izquierda no es `null` ni `undefined`; en caso contrario devuelve `undefined` en lugar de lanzar un error. Lo usarás mucho con objetos (UT4) y con el DOM (UT6):

```
const usuario = null;
usuario.nombre;    // TypeError: Cannot read properties of null
usuario?.nombre;   // undefined, sin error
```

6.6. Operador condicional (ternario)

`condición ? valorSiCierto : valorSiFalso` es un `if` en forma de **expresión**: produce un valor y se puede usar dentro de una asignación o de una plantilla.

```
const edad = 17;
const tipo = edad >= 18 ? 'adulto' : 'menor';
console.log(`Entrada de ${tipo}: ${edad >= 18 ? 12 : 8} €`);
```

Úsalo para elegir entre **dos valores sencillos**. Si necesitas más de dos ramas o ejecutar varias sentencias, usa `if`: los ternarios anidados son difíciles de leer.

6.7. Otros operadores

- `typeof`: tipo de un valor (apartado 4.1).
- **Operadores de cadena**: `+` concatena; las plantillas literales suelen ser más claras.
- **Operadores a nivel de bit** (`&`, `|`, `^`, `~`, `<<`, `>>`): trabajan con la representación binaria. Se usan poco en el desarrollo web; basta con saber que existen.
- **Operador coma (`,`) y `void`**: existen, pero no los usaremos de momento.

6.8. Precedencia

Cuando una expresión mezcla operadores, se aplican por **precedencia** (de más a menos): paréntesis → `**` → `*`, `/`, `%` → `+`, `-` → comparaciones → `===`, `!==` → `&&` → `||`, `??` → ternario → asignación. Con la misma precedencia, se evalúan de izquierda a derecha (salvo `**` y la asignación, que van de derecha a izquierda).

```
2 + 3 * 4;          // 14, no 20
(2 + 3) * 4;        // 20
5 > 3 && 2 > 1;     // true
!true || true;      // true  → ! se aplica antes que ||
```

No memorices la tabla completa: **usa paréntesis** siempre que una expresión pueda leerse de dos maneras. Cuestan poco y evitan errores. De hecho, JavaScript **obliga** a usarlos si mezclas `??` con `&&` o `||` en la misma expresión.

7. Estructuras de decisión

Hasta ahora, los programas ejecutaban las sentencias de arriba abajo. Las **estructuras de control** cambian ese flujo: las de **decisión** eligen qué bloque ejecutar y las de **repetición** (apartado 8) ejecutan un bloque varias veces.

7.1. `if`, `else` e `if...else if`

```
const temperatura = Number(prompt('Temperatura (°C):'));

if (temperatura >= 30) {
  console.log('Hace calor');
} else if (temperatura >= 18) {
  console.log('Temperatura agradable');
} else if (temperatura >= 5) {
  console.log('Hace fresco');
} else {
  console.log('Hace frío');
}
```

La condición se evalúa como booleano (recuerda los valores *falsy*). Las ramas se comprueban **en orden** y solo se ejecuta **la primera** que se cumple: en el ejemplo, con 35 grados no se comprueba

el resto. Por eso el orden importa: si pusieras `>= 5` primero, un día de 35 grados saldría «Hace fresco».

Convenciones del módulo: **siempre con llaves**, aunque el bloque tenga una sola sentencia (evita errores al añadir líneas más tarde), y la sentencia dentro del bloque en su propia línea.

7.2. Condiciones compuestas y anidadas

Combina condiciones con `&&`, `||` y `!`; anida un `if` dentro de otro solo cuando de verdad haya una decisión dentro de otra. Muchos `if` anidados se pueden aplanar:

```
// Difícil de leer
if (edad >= 18) {
  if (tieneCarnet) {
    console.log('Puede conducir');
  }
}

// Mejor
if (edad >= 18 && tieneCarnet) {
  console.log('Puede conducir');
}
```

Un patrón muy útil es la **cláusula de guarda**: comprobar primero los casos inválidos y salir, de modo que el resto del código no necesite anidarse:

```
const litros = Number(prompt('Litros:'));

if (Number.isNaN(litros) || litros < 0) {
  console.error('Introduce un número positivo');
} else {
  const precio = litros * 1.45;
  console.log(`Total: ${precio.toFixed(2)} €`);
}
```

7.3. `switch`

Cuando comparas **una misma expresión** con varios valores concretos, `switch` es más legible que una cadena de `else if`:

```
const opcion = prompt('Elige: 1 Alta · 2 Listado · 3 Salir');

switch (opcion) {
  case '1':
    console.log('Alta de un elemento');
    break;
  case '2':
    console.log('Listado');
    break;
  case '3':
    console.log('Hasta pronto');
    break;
  default:
    console.log('Opción no válida');
}
```

Detalles que hay que saber:

- La comparación es **estricta** (`===`): `case 1` no coincide con `'1'`. Fíjate en que `prompt` devuelve texto, por eso los `case` del ejemplo llevan comillas.
- Cada caso termina con **`break`**. Si lo olvidas, la ejecución **continúa en el caso siguiente** (*fall-through*). Es un error tan común que ESLint lo detecta.
- El *fall-through* se usa a propósito para **agrupar casos** dejando varios `case` seguidos sin sentencias:

```
switch (mes) {
  case 1: case 3: case 5: case 7: case 8: case 10: case 12:
    dias = 31;
    break;
  case 4: case 6: case 9: case 11:
    dias = 30;
    break;
  case 2:
    dias = esBisiesto ? 29 : 28;
    break;
  default:
    dias = 0;
}
```

- `default` recoge lo que no coincide con ningún caso. No es obligatorio, pero conviene ponerlo.

¿`if` o `switch`? `switch` cuando comparas un valor con constantes (menús, códigos, categorías). `if` para rangos (`>=`, `<`) y condiciones compuestas.

8. Bucles

Un **bucle** repite un bloque mientras se cumple una condición. Cada repetición es una **iteración**. Todo bucle tiene tres piezas: una **inicialización**, una **condición** de continuación y una **actualización** que acerca el bucle a su final. Si la actualización falta o no cambia la condición, el bucle es **infinito** y la página se cuelga (si te pasa, cierra la pestaña).

8.1. `while`: mientras se cumpla

Se usa cuando **no sabes de antemano** cuántas iteraciones habrá.

```
let intentos = 0;
let respuesta = '';

while (respuesta !== 'si' && intentos < 3) {
  respuesta = prompt('¿Continuar? (si/no)').toLowerCase();
  intentos++;
}
```

8.2. `do...while`: al menos una vez

Igual que `while`, pero la condición se comprueba **después** de cada iteración, así que el bloque se ejecuta como mínimo una vez. Es el bucle natural para **pedir un dato hasta que sea válido** y para los **menús**:

```
let numero;

do {
  numero = Number(prompt('Escribe un número entre 1 y 10:'));
} while (Number.isNaN(numero) || numero < 1 || numero > 10);

console.log(`Has elegido ${numero}`);
```

8.3. `for`: un número conocido de veces

Reúne en una línea la inicialización, la condición y la actualización. Es el bucle para «repetir N veces» o recorrer posiciones.

```
for (let i = 1; i <= 10; i++) {
  console.log(`7 × ${i} = ${7 * i}`);
}

for (let i = 10; i >= 0; i -= 2) {  // cuenta atrás de dos en dos
  console.log(i);
}
```

La variable de control se declara con `let` (así solo existe dentro del bucle). Dos errores clásicos: **modificar `i` dentro del cuerpo** y **equivocarse en un límite** (`<` frente a `<=`), que provoca una iteración de más o de menos (*off-by-one*).

8.4. `for...of`: recorrer una colección

Recorre **los valores** de cualquier objeto iterable: una cadena, un array (UT4), un `Map`… Es más claro que un `for` con índice cuando no necesitas la posición.

```
const palabra = 'canarias';
let vocales = 0;

for (const letra of palabra) {
  if ('aeiou'.includes(letra)) {
    vocales++;
  }
}
console.log(`${palabra} tiene ${vocales} vocales`);

const notas = [7, 4.5, 9, 6];   // un array: lo estudiarás en la UT4
let suma = 0;
for (const nota of notas) {
  suma += nota;
}
console.log(`Media: ${suma / notas.length}`);
```

Si necesitas también la posición, usa `for (const [indice, valor] of notas.entries())` (lo verás en la UT4).

8.5. `for...in`: recorrer las claves de un objeto

Recorre **los nombres de las propiedades** de un objeto. Lo verás en la UT4; de momento, solo recuerda que **no se usa para arrays** (recorre índices como texto y en un orden no garantizado): para eso está `for...of`.

```
const receta = { nombre: 'Gofio escaldado', tiempo: 15, dificultad: 'baja' };

for (const clave in receta) {
  console.log(`${clave}: ${receta[clave]}`);
}
```

8.6. `break` y `continue`

- `break` termina el bucle inmediatamente.
- `continue` salta a la siguiente iteración, sin ejecutar el resto del cuerpo.

```
// Primer divisor de un número (distinto de 1)
const n = 91;
let divisor = 2;
while (divisor < n) {
  if (n % divisor === 0) {
    break;              // encontrado: salimos
  }
  divisor++;
}
console.log(divisor === n ? `${n} es primo` : `${n} es divisible por ${divisor}`);

// Imprimir solo los impares
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) {
    continue;           // par: pasamos al siguiente
  }
  console.log(i);
}
```

Úsalos con moderación: un bucle con muchos `break` es difícil de seguir. A menudo, una condición bien escrita en el `while` los hace innecesarios. Con bucles anidados, `break` solo sale del bucle **interior**; existen las **etiquetas** (`exterior: for (...)` y `break exterior;`), pero es preferible reestructurar el código.

8.7. Bucles anidados

Un bucle dentro de otro: por cada iteración del exterior, el interior se ejecuta completo. Sirven para tablas, cuadrículas y combinaciones.

```
// Tablas de multiplicar del 1 al 3
for (let tabla = 1; tabla <= 3; tabla++) {
  console.group(`Tabla del ${tabla}`);
  for (let i = 1; i <= 10; i++) {
    console.log(`${tabla} × ${i} = ${tabla * i}`);
  }
  console.groupEnd();
}

// Triángulo de asteriscos
let figura = '';
for (let fila = 1; fila <= 5; fila++) {
  figura += '*'.repeat(fila) + '\n';
}
console.log(figura);
```

Cuidado con el coste: dos bucles de 1 000 iteraciones anidados son un millón de operaciones. En el navegador, un bucle largo **bloquea la página** mientras dura (recuerda el hilo único de la UT1).

8.8. Patrones habituales con bucles

Casi todos los ejercicios de esta unidad combinan cuatro patrones:

Patrón	Idea	Esqueleto
Contador	Cuántos elementos cumplen algo	`let cuenta = 0; … if (cond) { cuenta++; }`
Acumulador	Sumar o concatenar	`let suma = 0; … suma += valor;`
Máximo / mínimo	Guardar el mejor visto hasta ahora	`let max = -Infinity; … if (v > max) { max = v; }`
Bandera	Saber si ocurrió algo	`let encontrado = false; … if (cond) { encontrado = true; break; }`

```
// Ejemplo: estadísticas de cinco notas introducidas por el usuario
let suma = 0;
let maxima = -Infinity;
let aprobados = 0;
let hayMatricula = false;

for (let i = 1; i <= 5; i++) {
  const nota = Number(prompt(`Nota ${i}:`));
  suma += nota;                          // acumulador
  if (nota > maxima) { maxima = nota; } // máximo
  if (nota >= 5) { aprobados++; }       // contador
  if (nota === 10) { hayMatricula = true; } // bandera
}

console.log(`Media: ${(suma / 5).toFixed(2)} · Máxima: ${maxima}`);
console.log(`Aprobados: ${aprobados} · ¿Alguna matrícula? ${hayMatricula ? 'sí' : 'no'}`);
```

8.9. ¿Qué bucle elijo?

Situación	Bucle
Sé cuántas veces repetir, o necesito un índice	`for`
Recorrer los elementos de una cadena o colección	`for…of`
Repetir hasta que ocurra algo, sin saber cuántas veces	`while`
Pedir datos hasta que sean válidos; menús	`do…while`
Recorrer las propiedades de un objeto	`for…in` (UT4)

9. Comentarios y documentación

9.1. Comentarios

```
// Comentario de una línea

/*
  Comentario
  de varias líneas
*/

const precio = 100; // también al final de una línea
```

Un buen comentario explica **por qué**, no **qué**: el código ya dice lo que hace. Compara:

```
i++; // incrementa i                          ← inútil: repite el código
descuento = 0.15; // campaña de octubre, ver correo de dirección del 2/10  ← útil
```

Otros usos legítimos: avisar de una limitación (`// TODO: validar la fecha`), explicar una decisión poco obvia o desactivar código temporalmente mientras depuras (pero **no dejes código comentado** en las entregas: para eso está Git). En VS Code, **Ctrl + ç** (o **Ctrl + /**) comenta y descomenta la selección.

9.2. Documentación con JSDoc

JSDoc es una convención para documentar código JavaScript con comentarios especiales que empiezan por `/**`. Describe qué hace una función, qué parámetros recibe (con su tipo) y qué devuelve. Aunque las funciones se estudian en la UT4, en esta unidad ya escribirás algunas pequeñas, y desde el primer día las documentaremos así:

```
/**
 * Calcula el precio final aplicando el IGIC.
 * @param {number} base - Precio sin impuestos, en euros.
 * @param {number} [tipo=0.07] - Tipo impositivo (0.07 = 7 %).
 * @returns {number} Precio con impuestos, redondeado a 2 decimales.
 */
function precioConIgic(base, tipo = 0.07) {
  return Math.round(base * (1 + tipo) * 100) / 100;
}
```

Etiqueta	Para qué
Primera línea	Descripción breve, en una frase
`@param {tipo} nombre - descripción`	Cada parámetro. `[nombre]` si es opcional, `[nombre=valor]` si tiene valor por defecto
`@returns {tipo} descripción`	Lo que devuelve

`@throws {TipoError} descripción`	Errores que puede lanzar
`@example`	Un ejemplo de uso
`@type {tipo}`	Tipo de una variable o constante

Los tipos van entre llaves: `{number}`, `{string}`, `{boolean}`, `{number[]}` (array de números), `{string|null}` (texto o nulo), `{Object}`. Dos beneficios inmediatos: VS Code **muestra la documentación** al pasar el ratón por la función y **detecta** si la llamas con un tipo equivocado, y en la UT5 verás que JSDoc es el puente natural hacia TypeScript. Además, con la herramienta `jsdoc` se genera una web de documentación a partir de estos comentarios.

> ***Consejo.*** *Escribe `/**` justo encima de una función y pulsa Intro: VS Code genera el esqueleto con los `@param` de esa función.*

10. Estilo de código: Prettier y ESLint

10.1. Por qué importa el estilo

El código se escribe una vez y se lee muchas. Un estilo uniforme (misma sangría, mismas comillas, mismos nombres) hace que cualquier fichero del equipo se lea igual y que los errores salten a la vista. En la profesión, el estilo no se discute en cada línea: se **configura una herramienta** y se cumple. Nosotros usamos dos, complementarias:

Herramienta	Qué hace	Cuándo actúa
Prettier	**Formatea**: sangría, comillas, punto y coma, longitud de línea, espacios	Al guardar (ya lo configuraste en la UT1)
ESLint	**Analiza**: variables sin usar, `==`, `var`, `case` sin `break`, código inalcanzable…	Mientras escribes (subrayado) y con `npx eslint .`

Prettier decide **cómo se ve** el código; ESLint decide **si está bien escrito**. Ninguna de las dos ejecuta el programa: son **análisis estático**, detectan problemas antes de ejecutar. Precisamente lo que a un lenguaje de guiones le falta respecto a un compilador.

10.2. Convenciones del módulo

Estas son las normas que aplicarán las herramientas y que se valoran en las rúbricas:

- Sangría de **2 espacios**, **comillas simples** y **punto y coma** al final de cada sentencia (Prettier).
- `const` por defecto, `let` si hace falta reasignar, **nunca** `var`.
- Comparaciones con `===` y `!==`.
- **Llaves** siempre en `if`, `else`, `for` y `while`, aunque haya una sola sentencia.
- Nombres en `camelCase`, descriptivos, sin tildes; constantes de valor fijo en `MAYUSCULAS`.

- Una sentencia por línea; líneas de como máximo 80–100 caracteres.
- Sin variables ni código sin usar, sin `console.log` de depuración olvidados en las entregas.
- Cada función con su comentario **JSDoc**.

10.3. Instalar ESLint en el repositorio de ejercicios

ESLint se instala **por proyecto** con npm, y su configuración se guarda en el repositorio para que todo el equipo use las mismas reglas. Desde la raíz de `dew-ejercicios`:

```
npm init -y                        # crea package.json
npm init @eslint/config@latest     # asistente de configuración
```

El asistente hace unas preguntas; responde así: *How would you like to use ESLint?* → **To check syntax and find problems** · *What type of modules?* → **JavaScript modules (import/export)** · *Which framework?* → **None** · *Does your project use TypeScript?* → **No** · *Where does your code run?* → **Browser** (y también **Node**) · *Install now?* → **Yes**, con **npm**.

Al terminar tendrás una carpeta `node_modules/` (ya está en el `.gitignore`), un `package.json` con la dependencia y un fichero `eslint.config.js` parecido a este, al que añadimos unas reglas propias:

```
// eslint.config.js
import js from '@eslint/js';
import globals from 'globals';
import { defineConfig } from 'eslint/config';

export default defineConfig([
  {
    files: ['**/*.js'],
    plugins: { js },
    extends: ['js/recommended'],
    languageOptions: {
      ecmaVersion: 'latest',
      sourceType: 'script',
      globals: { ...globals.browser, ...globals.node },
    },
    rules: {
      'no-var': 'error',             // prohíbe var
      'prefer-const': 'warn',        // avisa si un let nunca se reasigna
      eqeqeq: ['error', 'always'],   // exige === y !==
      curly: ['error', 'all'],       // exige llaves en if, for, while…
      'no-unused-vars': 'warn',      // variables declaradas y no usadas
      'no-console': 'off',           // en esta unidad la consola es la salida
    },
  },
]);
```

Para ejecutarlo sobre todo el repositorio:

```
npx eslint .            # lista los problemas
npx eslint . --fix      # corrige los que se pueden corregir solos
```

La extensión **ESLint** de VS Code (la instalaste en la UT1) lee ese mismo fichero y subraya los problemas mientras escribes; al pasar el ratón ves la regla y una explicación. Si `npx eslint .` no da ningún mensaje, enhorabuena: no hay problemas.

> ***Nota.*** *`package.json` y `eslint.config.js` **sí** se suben al repositorio; `node_modules/` **no**. Quien clone el repositorio ejecuta `npm install` y obtiene las mismas herramientas.*

10.4. Tipos de mensajes

ESLint distingue **errores** (rojo: hay que corregirlos) y **avisos** (amarillo: conviene corregirlos). Cuando una regla no tenga sentido en un caso concreto puedes desactivarla en una línea con un comentario (`// eslint-disable-next-line no-unused-vars`), pero hazlo pocas veces y explica por qué.

11. Depuración y prueba del código

11.1. Tres clases de errores

Tipo	Cuándo aparece	Ejemplo	Cómo se detecta
De sintaxis	Al cargar el fichero: el programa no llega a ejecutarse	Paréntesis sin cerrar, `cosnt` en lugar de `const`	La consola muestra `SyntaxError` con la línea; ESLint y VS Code lo subrayan antes
De ejecución (excepciones)	Al ejecutar una sentencia concreta: el programa se detiene ahí	Usar una variable no declarada, llamar a un método de `null`	La consola muestra el error, el fichero y la línea
Lógicos	Nunca «salta»: el programa termina, pero el resultado es incorrecto	`<` en lugar de `<=`, `+` que concatena, condición al revés	Solo con **pruebas** y **depuración**

Los dos primeros son los fáciles: el navegador te dice dónde están. Los lógicos son los que consumen tiempo, y contra ellos están el depurador y las pruebas.

11.2. Leer un mensaje de error

Un mensaje de la consola tiene tres partes: el **tipo** de error, la **descripción** y la **pila de llamadas** con el fichero y la línea (haz clic en el enlace `app.js:12` y VS Code, o el panel Sources, te lleva a la línea). Los tipos más frecuentes en esta unidad:

Mensaje	Causa habitual
`SyntaxError: Unexpected token '}'`	Falta o sobra un paréntesis, llave o coma antes de esa posición
`SyntaxError: missing ) after argument list`	Paréntesis de una llamada sin cerrar
`ReferenceError: precio is not defined`	La variable no está declarada, tiene otro nombre (mayúsculas) o está en otro ámbito
`ReferenceError: Cannot access 'x' before initialization`	Usas una `let`/`const` antes de su línea (zona muerta temporal)
`TypeError: Assignment to constant variable`	Reasignas una `const`
`TypeError: Cannot read properties of null (reading 'textContent')`	Un `querySelector` no encontró el elemento (o el script no lleva `defer`)
`TypeError: x.toUpperCase is not a function`	`x` no es una cadena (probablemente es un número o `undefined`)
`RangeError: Maximum call stack size exceeded`	Recursión sin fin (UT4)
`Uncaught …` (prefijo)	El error no fue capturado por el programa: se detuvo la ejecución del script

11.3. Depurar con `console.log`

La técnica más simple: imprimir el valor de una variable en puntos clave para ver qué está pasando. Es válida, pero tiene límites: ensucia el código, hay que borrarla después y no permite «pararse» a mirar. Dos trucos que la mejoran:

```
console.log({ precio, unidades, total }); // entre llaves: muestra nombre y valor
console.table({ precio, unidades, total });
```

11.4. El depurador de las DevTools

El depurador permite **detener** el programa en una línea, mirar el valor de todas las variables y **avanzar paso a paso**. Está en el panel **Sources** (en Firefox, **Depurador**).

1. Abre el panel **Sources** y localiza tu fichero `.js` en el árbol de la izquierda (o con **Ctrl + P**).
2. Haz clic en el **número de una línea** para poner un **punto de ruptura** (*breakpoint*). Aparece una marca azul.
3. Recarga la página. La ejecución se **detiene** en esa línea antes de ejecutarla; la página se queda «pausada».
4. En el panel derecho, **Scope** muestra las variables de cada ámbito (Local, Script/Global) con sus valores actuales. Al pasar el ratón por una variable del código también ves su valor.
5. Avanza con los botones de la barra superior (o con el teclado):

Botón	Tecla	Qué hace
Resume (▶)	F8	Continúa hasta el siguiente punto de ruptura o el final
Step over (⤼)	F10	Ejecuta la línea actual y se para en la siguiente, sin entrar en las funciones
Step into (↓)	F11	Entra en la función que se llama en esta línea
Step out (↑)	Mayús + F11	Termina la función actual y vuelve a quien la llamó
Step	F9	Ejecuta la siguiente sentencia, sea la que sea

Otras piezas útiles del panel: **Watch** (expresiones que quieres vigilar, por ejemplo `total / unidades`), **Call Stack** (qué función llamó a cuál), **Breakpoints** (lista de todos, con casilla para desactivarlos), los **puntos de ruptura condicionales** (clic derecho sobre el número de línea → *Add conditional breakpoint*: `i === 57`, para detener un bucle solo en la iteración que te interesa) y **Pause on exceptions** (icono de pausa: se detiene justo donde salta un error).

También puedes escribir la sentencia `debugger;` en el código: cuando las DevTools están abiertas, el programa se detiene ahí como si hubiera un punto de ruptura. Es cómoda, pero **bórrala antes de entregar** (ESLint la marca).

> ***Consejo.*** *Mientras el programa está pausado, la* ***consola*** *funciona en el ámbito de la línea actual: puedes escribir el nombre de una variable local para ver su valor o ejecutar una expresión con ella. Es la forma más rápida de comprobar una hipótesis: «¿será que* `edad` *es un texto?» → escribe* `typeof edad`*.*

11.5. Método para depurar

1. **Reproduce** el fallo: ¿con qué datos de entrada ocurre? ¿Siempre o solo a veces?
2. **Lee el error** (si lo hay) y ve a la línea que indica. Si no hay error, es un fallo lógico.
3. **Formula una hipótesis**: «el total sale mal porque `precio` es texto».
4. **Compruébala** con un punto de ruptura antes de la línea sospechosa, mirando los valores en Scope o en la consola.

5. **Corrige** una cosa cada vez y **vuelve a probar** con los mismos datos.
6. Prueba también con **otros datos** para asegurarte de que no has roto otro caso.

11.6. Prueba manual: casos de prueba

Probar «a ver si funciona» con el primer valor que se te ocurre no basta. Antes de dar por terminado un programa, diseña una **tabla de casos de prueba** que cubra los casos normales, los **límites** y los **inválidos**, y ejecútala. Para un conversor de notas numéricas a texto (`< 5` suspenso, `5–6.99` aprobado…):

N.º	Entrada	Resultado esperado	Obtenido	✓
1	`7`	Notable	Notable	✓
2	`4.99`	Suspenso (límite inferior de aprobado)	Suspenso	✓
3	`5`	Aprobado (límite exacto)	Suspenso	✗ → error en `>` en lugar de `>=`
4	`10`	Sobresaliente (límite superior)		
5	`-1`	Mensaje de error		
6	`'hola'`	Mensaje de error		
7	*(cancelar)*	Mensaje de error		

Los valores en los **límites** (`5`, `4.99`, `10`, `0`) son los que más errores destapan. La tabla se entrega junto con el programa: forma parte del trabajo. En la UT4 automatizaremos estas pruebas con Vitest, y verás que la tabla se convierte casi directamente en código.

11.7. Control de versiones como herramienta de prueba

Un uso poco evidente de Git: si un programa funcionaba y ahora no, `git diff` te muestra exactamente qué cambió desde el último commit. Por eso interesa hacer un commit cada vez que un ejercicio **funciona**: es un punto seguro al que volver.

12. Resumen

- JavaScript moderno se ejecuta en la **consola**, en un **fichero enlazado** con `defer` o en **Node.js**. El **modo estricto** evita errores silenciosos.
- Declara con `const` por defecto y `let` si hay que reasignar; `var` solo se lee en código antiguo. Las declaraciones se registran antes de ejecutar (*hoisting*), pero `let` y `const` no se pueden usar antes de su línea.
- Tres **ámbitos**: global, de función y de bloque. `let` y `const` respetan los bloques; `var` no.

- Las variables no tienen tipo; los valores sí. **Primitivos**: `number`, `bigint`, `string`, `boolean`, `undefined`, `null`, `symbol`. Los objetos (incluidos arrays y funciones) son **referencias**. `typeof` dice el tipo.
- Los decimales no son exactos (`0.1 + 0.2`), `NaN` es el resultado de operaciones imposibles y las cadenas son **inmutables**.
- JavaScript **convierte tipos en silencio**: `+` concatena si hay una cadena; los demás operadores convierten a número. Convierte tú con `Number()`, `String()` y `Boolean()`, y compara siempre con `===`.
- Siete valores ***falsy***: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. Todo lo demás es *truthy*.
- `&&` y `||` cortocircuitan y devuelven un operando; `??` solo sustituye `null` y `undefined`; `?.` evita errores al acceder a algo que puede no existir; el **ternario** elige entre dos valores.
- `if...else if` para rangos y condiciones compuestas; `switch` para comparar con valores concretos (con `break`).
- `for` para repetir N veces, `for...of` para recorrer, `while` cuando no se sabe cuántas veces, `do...while` para pedir datos hasta que sean válidos. `break` sale, `continue` salta. Patrones: contador, acumulador, máximo, bandera.
- Comenta el **porqué**, no el qué, y documenta cada función con **JSDoc**.
- **Prettier** formatea y **ESLint** detecta problemas sin ejecutar. La configuración vive en el repositorio.
- Errores **de sintaxis**, **de ejecución** y **lógicos**. El **depurador** (puntos de ruptura, paso a paso, Scope, Watch) y una **tabla de casos de prueba** con valores límite son la forma profesional de encontrarlos.

13. Glosario

Término	Definición
Sentencia	Instrucción completa del programa; termina en punto y coma.
Expresión	Combinación de valores, variables y operadores que produce un valor.
Bloque	Conjunto de sentencias entre llaves `{ }`; crea un ámbito para `let` y `const`.
Modo estricto	Variante del lenguaje (`'use strict'`) que convierte en error varios comportamientos peligrosos.
Variable	Nombre asociado a un valor que puede cambiar (`let`).
Constante	Nombre asociado a un valor que no se puede reasignar (`const`).
Hoisting	Registro de las declaraciones antes de ejecutar el código.

Zona muerta temporal	Tramo entre el inicio del ámbito y la línea de declaración de una `let`/`const`, en el que no se puede usar.
Ámbito	Región del programa en la que una variable existe: global, de función o de bloque.
Sombreado	Declarar en un ámbito interior una variable con el mismo nombre que otra exterior.
Tipado dinámico	Las variables no tienen tipo fijo; el tipo es del valor que contienen.
Tipado débil	El lenguaje convierte automáticamente entre tipos cuando una operación lo requiere.
Primitivo	Valor simple e inmutable que se copia por valor (`number`, `string`, `boolean`…).
Referencia	Valor (objeto) del que las variables guardan una dirección; se comparte, no se copia.
Coerción	Conversión implícita de tipos que realiza el lenguaje.
Truthy* / *falsy	Valores que se evalúan como `true` o `false` en una condición.
`NaN`	«Not a Number»: resultado numérico de una operación imposible.
Cortocircuito	Evaluación de `&&` y `||` que se detiene en cuanto conoce el resultado.
Coalescencia nula	Operador `??`: devuelve el operando derecho solo si el izquierdo es `null` o `undefined`.
Encadenamiento opcional	Operador `?.`: accede a una propiedad solo si el valor existe.
Ternario	Operador `? :` que elige entre dos valores según una condición.
Fall-through	Continuación de la ejecución de un `case` al siguiente por falta de `break`.
Iteración	Cada ejecución del cuerpo de un bucle.
Off-by-one	Error de una iteración de más o de menos por un límite mal escrito.
Iterable	Valor que se puede recorrer con `for…of` (cadenas, arrays, `Map`, `Set`…).
JSDoc	Convención de comentarios `/** … */` para documentar funciones, parámetros y tipos.

Análisis estático	Revisión del código sin ejecutarlo (ESLint, Prettier, TypeScript).
Linter	Herramienta que detecta errores y malas prácticas en el código (ESLint).
Punto de ruptura	Línea en la que el depurador detiene la ejecución.
Caso de prueba	Entrada concreta con su resultado esperado, para verificar un programa.
Valor límite	Entrada situada justo en la frontera de una condición (`5` en `>= 5`).

14. Actividades de la unidad

Todas las actividades se hacen en tu repositorio `DEW-ejercicios`, dentro de la carpeta `ut02`, una subcarpeta por actividad (`a1-consola`, `a2-variables`…), cada una con el `index.html` mínimo del apartado 1.7 y su `app.js`. Haz **un commit por actividad terminada** (o más, si es larga) y `push` al acabar cada sesión. Salvo que se indique lo contrario, las actividades son individuales. La A8 es el **producto evaluable** de la unidad y la A9 es la tarea del proyecto.

A1. La consola es tu terminal (CE a, h)

Objetivo: ejecutar JavaScript en los tres entornos y comprobar el efecto del modo estricto.

- Abre la consola de las DevTools en cualquier página y evalúa, una a una, estas expresiones anotando el resultado: `2 ** 10`, `'DAW'.length`, `typeof 3.5`, `typeof '3.5'`, `10 / 3`, `10 % 3`, `'10' + 3`, `'10' - 3`. Escribe en la consola una plantilla literal que muestre tu nombre y el año actual.
- Crea `ut02/a1-consola` con el `index.html` mínimo y un `app.js` que muestre con `console.log`, `console.warn`, `console.error` y `console.table` cuatro mensajes distintos, y agrupe dos de ellos con `console.group`. Ábrelo con Live Server y comprueba cómo se ve cada método en la consola.
- Ejecuta el mismo `app.js` con Node.js desde la terminal (`node app.js`). ¿Qué diferencias ves? Añade una línea con `alert('hola')` y vuelve a ejecutarlo en Node. Anota el error y explícalo.
- Añade al principio de `app.js` una asignación a una variable no declarada (`resultado = 42;`) y comprueba que funciona. Añade ahora `'use strict';` en la primera línea y recarga. Anota el error.
- Crea en **Sources** → **Snippets** un fragmento llamado `pruebas` con las expresiones del punto 1 y ejecútalo.

Entrega: la carpeta `a1-consola` y un `NOTAS.md` con los resultados y las respuestas de los puntos 1, 3 y 4.

A2. Laboratorio de variables y ámbitos (CE b, c)

Objetivo: predecir y comprobar el comportamiento de `var`, `let` y `const` y de los ámbitos.

Para cada uno de los fragmentos siguientes, **antes de ejecutarlo**, escribe en `PREDICCIONES.md` qué crees que mostrará la consola (o qué error dará). Después ejecútalo, anota el resultado real y, si no coincide con tu predicción, explica el porqué con los apartados 2 y 3.

```
// Fragmento 1
console.log(a);
var a = 5;
console.log(a);

// Fragmento 2
console.log(b);
let b = 5;

// Fragmento 3
const c = 1;
c = 2;

// Fragmento 4
if (true) {
  var d = 'var';
  let e = 'let';
}
console.log(d);
console.log(e);

// Fragmento 5
for (var i = 0; i < 3; i++) {}
for (let j = 0; j < 3; j++) {}
console.log(i);
console.log(j);

// Fragmento 6
const nivel = 'exterior';
{
  const nivel = 'interior';
  console.log(nivel);
}
console.log(nivel);

// Fragmento 7
let x = 1;
{
  console.log(x);
  let x = 2;
}

// Fragmento 8
function calcular() {
  const secreto = 42;
  return secreto * 2;
}
console.log(calcular());
console.log(secreto);
```

Los fragmentos que dan error detienen el script: ejecútalos por separado (comenta los demás, o usa un snippet por fragmento).

Después, escribe un programa `app.js` de al menos 15 líneas que **declare correctamente** (con `const` o `let`, según corresponda, y nombres en `camelCase` descriptivos) los datos de una entidad de tu proyecto: por ejemplo, para una receta, su nombre, número de comensales, tiempo en minutos, si es vegetariana, categoría y valoración. Muestra los datos con una plantilla literal y comprueba con `typeof` que cada dato tiene el tipo adecuado.

Entrega: `PREDICCIONES.md` con la tabla (fragmento · predicción · resultado · explicación) y `app.js`.

A3. Laboratorio de tipos y conversiones (CE b, d)

Objetivo: reconocer y comprobar las peculiaridades de JavaScript en las conversiones entre tipos.

1. Como en A2, predice y después comprueba el resultado de estas expresiones, y clasifícalas en «esperable» o «sorprendente»: `'5' + 3`, `'5' - 3`, `'5' * '2'`, `1 + 2 + '3'`, `'1' + 2 + 3`, `true + true`, `'hola' * 2`, `null + 1`, `undefined + 1`, `0.1 + 0.2 === 0.3`, `10 / 0`, `'10' < '9'`, `10 < '9'`, `5 == '5'`, `5 === '5'`, `0 == ''`, `'' == '0'`, `null == undefined`, `null === undefined`, `NaN === NaN`, `Boolean('false')`, `Boolean(0)`, `Boolean('0')`, `Boolean([])`, `Number('')`, `Number(' 12 ')`, `Number('12px')`, `parseInt('12px')`, `parseInt('3.9')`, `parseFloat('3.9')`.
2. Completa la tabla del apartado 5.5 con estas filas y compruébalas: `' '` (un espacio), `'12.5'`, `-0`, `Infinity`, `[]`, `[7]`.
3. Escribe un programa que pida con `prompt` un número y muestre: el texto recibido, su `typeof`, el resultado de `Number()`, `parseInt()` y `parseFloat()`, y si es o no un número válido (`Number.isNaN`). Pruébalo con `42`, `3,5` (con coma), `3.5`, `12px`, una cadena vacía, `hola` y Cancelar. Anota los resultados.
4. Explica en cinco líneas, con tus palabras, por qué **`prompt('Edad:') + 1`** no hace lo que parece y cómo se corrige.

Entrega: `RESULTADOS.md` con las tablas de los puntos 1 y 2, la tabla de pruebas del punto 3 y la explicación del punto 4, y el `app.js` del punto 3.

A4. Conversores y calculadoras (CE b, d, g)

Objetivo: escribir programas secuenciales completos con entrada, cálculo y salida, y documentarlos.

Cada programa en su fichero (`conversor.js`, `imc.js`, `nomina.js`), todos enlazados desde el mismo `index.html` (varios `<script defer>`), y cada uno con un comentario de cabecera que indique qué hace, tu nombre y la fecha. Convierte siempre las entradas con `Number()`.

1. **Conversor de unidades.** Pide una cantidad en kilómetros y muestra su equivalente en metros, millas (1 km = 0,621371 mi) y millas náuticas (1 km = 0,539957 NM), con dos decimales. Amplía: pide grados Celsius y muestra Fahrenheit (`F = C × 9/5 + 32`) y Kelvin.
2. **Tiempo.** Pide un número de segundos y muéstralo como `h:mm:ss` (usa `%` y `Math.trunc`; rellena con ceros a la izquierda con `String(n).padStart(2, '0')`).

3. **IMC.** Pide peso (kg) y altura (cm) y muestra el índice de masa corporal (`peso / altura_en_metros²`) con un decimal. Sin decisiones todavía: solo el número.
4. **Nómina simplificada.** Pide el salario bruto anual y el número de pagas (12 o 14). Muestra el bruto mensual, la retención de IRPF aplicando un tipo fijo del 15 %, la cotización a la Seguridad Social del 6,47 % y el neto mensual, todo con dos decimales y en una tabla de la consola (`console.table`).
5. **Precio con IGIC.** Pide un precio sin impuestos y un tipo de IGIC (por defecto, si el usuario cancela, 7 % usando `??`). Muestra el importe del impuesto y el total. Documenta con JSDoc una función `precioConIgic(base, tipo)` como la del apartado 9.2 y úsala.

Entrega: la carpeta `a4-calculadoras` con los ficheros y, para cada programa, una tabla de **al menos tres casos de prueba** (entrada · esperado · obtenido) en `PRUEBAS.md`.

A5. Decisiones (CE e, g)

Objetivo: utilizar `if…else if`, operadores lógicos, el ternario y `switch` para crear bloques de sentencias.

1. **Clasificador de IMC.** Amplía `imc.js` (cópialo a esta carpeta): bajo peso (< 18,5), normal (18,5–24,9), sobrepeso (25–29,9), obesidad (≥ 30). Antes, valida que peso y altura sean números positivos; si no, muestra un error y no calcules nada (cláusula de guarda).
2. **Calificaciones.** Pide una nota de 0 a 10 (admite decimales) y muestra: insuficiente (< 5), suficiente (5–5,99), bien (6–6,99), notable (7–8,99), sobresaliente (9–10). Fuera de rango o no numérico: error. Escribe la **tabla de casos de prueba** con los valores límite (4,99 · 5 · 5,99 · 6 · 8,99 · 9 · 10 · 10,01 · -1 · texto).
3. **Año bisiesto.** Un año es bisiesto si es divisible por 4, salvo que sea divisible por 100, a no ser que también lo sea por 400. Resuélvelo con **una sola condición** usando `&&`, `||` y `%`, y comprueba con 1900 (no), 2000 (sí), 2024 (sí), 2026 (no).
4. **Días del mes.** Pide mes (1–12) y año y muestra cuántos días tiene, con un `switch` que agrupe casos y reutilice el cálculo de bisiesto.
5. **Tarifa de entrada.** Un museo cobra 12 €; gratis para menores de 6 años y mayores de 65; 50 % de descuento de 6 a 17 años o con carnet de estudiante; los miércoles es gratis para todos. Pide edad, si tiene carnet (`si`/`no`) y día de la semana, y muestra el precio. Escribe primero, en un comentario, el **orden** en que comprobarás las condiciones y por qué. Usa un ternario para el mensaje final («Entrada gratuita» / «Precio: … €»).
6. **Semáforo con `switch`.** Pide un color (`rojo`, `ambar`, `verde`) en cualquier combinación de mayúsculas y minúsculas (usa `toLowerCase()` y `trim()`) y muestra la acción. Cualquier otro texto: «Color desconocido». Después, quita **a propósito** un `break` y documenta en un comentario qué ocurre.

Entrega: carpeta `a5-decisiones` con un fichero por ejercicio y `PRUEBAS.md` con las tablas de casos de los ejercicios 2, 3 y 5.

A6. Bucles (CE f, g)

Objetivo: utilizar los distintos bucles, verificar su funcionamiento y aplicar los patrones de contador, acumulador, máximo y bandera.

Bloque 1 · `for`

1. Muestra la tabla de multiplicar del número que indique el usuario, del 1 al 10, en el formato `7 × 3 = 21`.
2. Muestra los números del 1 al 100 en una sola línea de la consola (acumula en una cadena y muestra al final), y después solo los múltiplos de 3 o de 5, con `continue`.
3. Calcula la suma y la media de los números del 1 al N (N lo indica el usuario). Comprueba con N = 100 (suma 5050).
4. **Serie.** Muestra los 15 primeros términos de la sucesión de Fibonacci (1, 1, 2, 3, 5, 8…).
5. **Potencias.** Muestra 2^0, 2^1, … 2^{10} sin usar `**` ni `Math.pow` (acumulador multiplicativo).

Bloque 2 · `while` y `do...while`

6. **Entrada válida.** Pide un número entre 1 y 20 hasta que el usuario introduzca uno válido (numérico y en rango). Cuenta los intentos y muéstralos al final.
7. **Adivina el número.** Genera un número secreto entre 1 y 100 (`Math.floor(Math.random() * 100) + 1`) y pide intentos indicando «mayor» o «menor» hasta acertar o agotar 7 intentos. Muestra el número de intentos usados.
8. **Suma hasta cero.** Pide números y acumula su suma hasta que el usuario introduzca 0. Muestra la suma, cuántos números hubo, el mayor y el menor (patrones acumulador, contador y máximo/mínimo).
9. **Menú.** Muestra con `prompt` un menú con tres opciones y «0 Salir», y repite hasta que se elija 0 (`do…while` + `switch`). Cada opción, de momento, solo muestra un mensaje.

Bloque 3 · `for...of` y bucles anidados

10. **Contador de vocales.** Pide una frase y cuenta vocales, consonantes y espacios con `for…of`. Ignora mayúsculas.
11. **Palíndromo.** Indica si una palabra se lee igual al derecho y al revés, construyendo la palabra invertida con un bucle (sin usar `reverse`).
12. **Figuras.** Con bucles anidados y `repeat` **prohibido**, dibuja en la consola un cuadrado hueco de N×N asteriscos y un triángulo rectángulo de N filas. Muestra el resultado con un único `console.log` de una cadena con `\n`.
13. **Primos.** Muestra los números primos entre 2 y N (bucle exterior por candidatos; bucle interior que busca un divisor y sale con `break`; bandera `esPrimo`). Con N = 50 debe haber 15 primos.
14. **Tablas completas.** Muestra las tablas del 1 al 10 agrupadas con `console.group`.

Para **cada ejercicio** anota en un comentario de cabecera qué bucle has elegido y por qué. Para los ejercicios 6, 7, 8 y 13 incluye una tabla de casos de prueba con al menos un caso límite.

Entrega: carpeta `a6-bucles` con un fichero por bloque (`for.js`, `while.js`, `anidados.js`) y `PRUEBAS.md`.

A7. Caza de errores (CE h)

Objetivo: localizar y corregir errores de sintaxis, de ejecución y lógicos con las herramientas de desarrollo.

Descarga del aula virtual la carpeta `a7-errores` con cinco programas que **no funcionan**. Cada uno tiene entre uno y tres errores. Para cada programa:

1. Ejecútalo con Live Server y anota el **mensaje exacto** de la consola, si lo hay, y la línea que señala.
2. Clasifica cada error como de **sintaxis**, de **ejecución** o **lógico**.
3. Para los errores lógicos, usa el **depurador**: pon un punto de ruptura, avanza paso a paso y haz una captura del panel **Scope** en el momento en que el valor de una variable revela el fallo.
4. Corrige el error, ejecuta `npx eslint .` y comprueba que no queda ningún aviso.
5. Verifica la corrección con la tabla de casos de prueba que acompaña a cada programa.

Después, en **`INFORME.md`**, rellena una tabla por programa: error · tipo · cómo lo encontraste (consola, ESLint, depurador, prueba) · corrección. Añade una reflexión de cinco líneas: ¿qué tipo de error te costó más encontrar y por qué?

Entrega: la carpeta corregida, `INFORME.md` y las capturas del depurador.

A8. Consola de utilidades (producto de la unidad; CE a–h)

Objetivo: integrar todo lo aprendido en un programa completo, documentado, con estilo verificado por ESLint y publicado en GitHub.

Crea `ut02/a8-utilidades` con un `index.html` y un `app.js`. El programa muestra un **menú** con `prompt` y se repite hasta que el usuario elija salir. Cada opción es un pequeño programa que ya has escrito en las actividades anteriores o una versión mejorada:

```
CONSOLA DE UTILIDADES · [tu nombre]
1. Conversor de unidades (km, °C, segundos)
2. Calculadora de IMC con clasificación
3. Precio con IGIC
4. Calificación de una nota
5. Generador de series (tabla, Fibonacci o primos, a elegir)
6. Estadísticas de una lista de números (hasta introducir 0)
7. Adivina el número
0. Salir
```

Requisitos:

	Requisito
R1	`'use strict'` al principio. Solo `const` y `let`. Nombres descriptivos en `camelCase`.
R2	Todas las entradas se convierten y **se validan** (número, rango, cancelación con `null`): el programa nunca muestra `NaN` ni se detiene por un error.
R3	Se usan, al menos una vez cada uno: `if…else if`, `switch`, `while` o `do…while`, `for`, `for…of`, el ternario y `??`.
R4	Cada utilidad está en su **función** documentada con **JSDoc** (`@param`, `@returns`), aunque sea sencilla; el menú llama a esas funciones.
R5	`npx eslint .` no da errores ni avisos. Prettier ha formateado el código.
R6	`PRUEBAS.md` con una tabla de casos de prueba para cada opción (mínimo 3 casos, con límites e inválidos), todos superados.
R7	`README.md` en la carpeta con qué hace el programa, cómo ejecutarlo y qué aprendiste.
R8	Al menos **5 commits** con mensajes significativos a lo largo de la actividad; `push` final antes del plazo.

Ampliación (opcional): opción 8, «Juego de la vida de una cuenta»: simula N meses de una cuenta con saldo inicial, ingreso mensual y gasto mensual, y muestra el mes en que el saldo se vuelve negativo, si ocurre.

Entrega: enlace al repositorio; el profesorado revisará la carpeta y el historial de commits. Se defiende oralmente en clase (dos o tres preguntas sobre el código).

A9. La lógica de tu proyecto (CE b, d, e, f, g)

Objetivo: aplicar la sintaxis del lenguaje a los datos de tu proyecto del curso.

En `DEW-proyecto`, en `js/app.js`, y usando solo lo aprendido en esta unidad (todavía sin arrays, objetos ni DOM, salvo el `#info` de la página inicial):

1. Declara con `const` y `let` los datos de **tres elementos** de tu entidad principal (cada propiedad en su variable, con el tipo adecuado: `const receta1Nombre = …`, `const receta1Tiempo = …`). Es incómodo; en la UT4 verás cómo lo arreglan los arrays y los objetos, y esa incomodidad es parte de la lección.
2. Escribe al menos **tres cálculos o decisiones** con sentido para tu tema: una media, un máximo, una clasificación por rangos (`if…else if`), una comprobación con operadores lógicos (por ejemplo, «apto para vegetarianos y en menos de 30 minutos»), un mensaje distinto según una categoría (`switch`)…
3. Escribe al menos **un bucle** con sentido: una cuenta atrás, una simulación de N periodos, una serie relacionada con tu tema…
4. Muestra los resultados en la consola con plantillas literales y `console.table`, y escribe **un resumen** en el párrafo `#info` de la página con `textContent`.
5. Documenta cada función con JSDoc, pasa ESLint y actualiza el `README.md` del proyecto con un apartado «UT2» que explique qué hace ahora la aplicación.

Entrega: commit y push en `DEW-proyecto`. Se revisa junto con A8.

15. Autoevaluación

1. ¿Qué diferencia hay entre `let` y `const`? ¿Y entre `let` y `var` dentro de un `for`?
2. ¿Qué muestra `console.log(x); let x = 3;`? ¿Y con `var`?
3. ¿Qué vale `'3' + 4 + 5`? ¿Y `3 + 4 + '5'`? ¿Y `'3' * '4'`?
4. Enumera los siete valores *falsy*. ¿Es `'0'` *truthy* o *falsy*?
5. ¿Por qué `0.1 + 0.2 === 0.3` es `false`? ¿Cómo comprobarías si un valor es `NaN`?
6. ¿Qué diferencia hay entre `==` y `===`? Pon un ejemplo en el que den resultados distintos.
7. ¿Qué devuelve `0 || 10`? ¿Y `0 ?? 10`? ¿Cuándo usarías cada uno?
8. Un `switch` compara con `case 1:` pero el valor viene de `prompt`. ¿Entra en ese caso? ¿Por qué?
9. ¿Qué ocurre si olvidas el `break` en un `case`? ¿Cuándo se hace a propósito?
10. ¿Qué bucle usarías para pedir una contraseña hasta que sea correcta? ¿Y para recorrer las letras de una palabra?
11. Escribe la condición de un `while` que se repita mientras `n` sea un número válido entre 1 y 10, sin usar `break`.
12. ¿Qué diferencia hay entre un error de ejecución y un error lógico? ¿Qué herramienta usarías para cada uno?
13. ¿Qué hace **Step over** y en qué se diferencia de **Step into**?
14. ¿Qué detecta ESLint que Prettier no detecta? ¿Y al revés?

15. Escribe el comentario JSDoc de una función `esBisiesto(anio)` que devuelve un booleano.

16. Recursos

- **MDN · Guía de JavaScript** (en español): https://developer.mozilla.org/es/docs/Web/JavaScript/Guide — gramática y tipos, control de flujo, bucles e iteración.
- **MDN · Referencia de operadores**: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Operators
- **javascript.info · Fundamentos** (en español): https://es.javascript.info/first-steps — variables, tipos, conversiones, operadores, condicionales y bucles, con ejercicios resueltos.
- **javascript.info · Depuración en el navegador**: https://es.javascript.info/debugging-chrome
- **Chrome DevTools · Depurar JavaScript**: https://developer.chrome.com/docs/devtools/javascript
- **ESLint · Primeros pasos**: https://eslint.org/docs/latest/use/getting-started
- **JSDoc · Guía de etiquetas**: https://jsdoc.app/
- **Tabla de igualdad de JavaScript** (visualización de `==`): https://dorey.github.io/JavaScript-Equality-Table/
- **Eloquent JavaScript** (libro gratuito; capítulos 1 y 2): https://eloquentjavascript.net/